

QuikSkill LMS

Frontend System Documentation & Migration Reference

[Routing](#) · [Pages by Role](#) · [Components](#) · [State](#) · [Realtime & Media](#)

Current Stack: **Vite + React 18 + React Router 6 + TypeScript + Tailwind**

Target Stack: **Next.js (App Router) + PostgreSQL backend**

Generated: June 2026 | Document 2 of 2 (Frontend)

Table of Contents

1. Overview & Tech Stack
2. Routing, Client Auth & Route Protection
3. Role Layouts & Sidebar Navigation
4. API / Data Layer, Env Vars & Socket.IO
5. State Management, Contexts & i18n
6. Complete Pages Catalog (by Role)
7. Key Feature Components
8. Real-time & Media Features
9. Vite → Next.js Migration Guidance

1. Overview & Tech Stack

The QuikSkill LMS web client is a single-page React application served by Vite. It renders seven role-specific experiences from one bundle, driven by the tenant's feature flags and the logged-in user's role. It talks to the NestJS REST API and two Socket.IO namespaces, and performs heavy in-browser work: SCORM playback, MediaPipe face-proctoring, resumable uploads, and PDF generation.

Concern	Library	Migration Note (Next.js)
Build / Dev	Vite 5	→ Next.js build pipeline
UI	React 18 + TypeScript	Retain; split into Server/Client Components
Routing	react-router-dom 6	→ Next.js App Router (file-based)
Styling	Tailwind CSS 3 + PostCSS	Retain as-is
HTTP	axios 1.6	Retain client util; or server actions
State	React Context API (no Redux/Zustand)	Wrap in root layout providers
Forms	react-hook-form	Retain
Realtime	socket.io-client	Client component + WS server
Drag & drop	@dnd-kit, react-beautiful-dnd	Retain (client only)
Media / Video	react-player, react-pdf	Retain (client only, dynamic import)
Proctoring	@mediapipe/face_detection, tasks-vision, camera_utils	Retain (client only)
Uploads	tus-js-client	Retain
PDF / Canvas	jspdf, html2canvas, pdfkit, canvas-confetti	Retain (client only)
Rich text	react-quill	Retain (dynamic import, SSR:false)
i18n	i18next + react-i18next + browser-languagedetector	→ next-intl / next-i18next
Charts	recharts	Retain (client only)
Icons / Toast	lucide-react, react-icons, react-hot-toast	Retain

Scale: ~89 page components across 8 role folders, ~60 feature components, 7 custom hooks, 3 context providers, 6 role layouts. Source tree: `src/{pages,components,contexts,hooks,utils,constants,i18n}` .

2. Routing, Client Auth & Route Protection

All routing lives in `src/App.tsx` using `<Routes>` / `<Route>`. Auth state is held in **browser storage** (no server session):

- `sessionStorage.access_token` — JWT attached to every API request.
- `sessionStorage.user` — JSON user object (role, secondaryRole, tenantId, ...).
- `localStorage.activeRole` — currently active role for multi-role users (set by `RoleSwitcher`).
- `localStorage.quickskill-theme`, `i18nextLng` — theme & language preferences.

Route Guards (3 wrappers in App.tsx)

Guard	Logic
<code>PrivateRoute</code>	Renders children only if <code>access_token</code> exists, else redirects to <code>/login</code> (spinner while checking).
<code>RoleProtectedRoute</code>	Checks <code>user.role</code> / <code>user.secondaryRole</code> / <code>activeRole</code> against an <code>allowedRoles</code> list; unauthorized users are bounced to their own dashboard via <code>getDashboardPath()</code> .
<code>RedirectIfAuthenticated</code>	Wraps <code>/login</code> — logged-in users are sent to their role dashboard so they never see the login page again.

Role → Landing Route

Role	Landing Path	Layout
<code>SUPER_ADMIN</code>	<code>/dashboard</code>	<code>SuperAdminLayout</code>
<code>TENANT_ADMIN</code>	<code>/tenant-dashboard</code>	<code>TenantAdminLayout</code>
<code>SUB_ADMIN</code>	<code>/sub-admin-dashboard</code>	<code>TenantAdminLayout</code> (minus sub-admin mgmt)
<code>MANAGER</code>	<code>/manager-dashboard</code>	<code>ManagerLayout</code>
<code>TEACHER</code>	<code>/teacher-dashboard</code>	<code>TeacherLayout</code>
<code>PARENT</code>	<code>/parent-dashboard</code>	<code>ParentLayout</code>
<code>LEARNER</code>	<code>/learner/dashboard</code>	<code>LearnerLayout</code> (+ fullscreen player routes)

Public routes: `/login`, `/setup-password`, `/auth/callback`, `/verify-certificate/:certificateId`. **Fullscreen (no layout):**

`/learner/course/:courseId` (UniversalLMSPlayer), `/learner/course/:courseId/view` (CourseViewerPage), `/learner/course/:courseId/legacy` (LockedCoursePlayer), `/exam/:examId/take` (ExamTakingPage).

3. Role Layouts & Sidebar Navigation

Each layout (in `src/components/layouts/`) renders a branded sidebar (logo + colors from `BrandingContext`) and an `<Outlet/>` . The `TenantAdminLayout` switches its nav between Corporate and School depending on `tenantType` from `FeatureContext` .

Layout	Sidebar Navigation Items
SuperAdminLayout	Overview: Organizations · Users · Approvals Content: Master Library · Question Bank · Certificates Platform: System Health · Audit · Profile Settings
TenantAdminLayout (Corporate)	Management: Learners · Course Assignment · Groups Operations: Compliance Content: Quiz Proctoring · Messages · Corporate Analytics Settings: Sub Admins · Sub-Admin Submissions · Create Course · Certificate Templates · My Submissions · Branding · Storage · Audit
TenantAdminLayout (School)	Management: Batches · Course Assignment · Teachers · Students · Parents Operations: Payouts · Credits Config · Escalations · Non-Teaching Tasks · Teacher Levels · Teacher Availability · Demo Analytics Content: Question Bank · Exams · Session Timestamps · Recordings · Messages · School Analytics Settings: Create Course · Certificate Templates · My Submissions · Branding · Storage · Audit · Video Config
ManagerLayout	Management: My Courses · My Team Certification: Certificates
TeacherLayout	Overview: My Classes Work: Attendance · Homework · Non-Teaching Tasks Growth: Payouts · My Level · Availability Support: Tutoring Requests · Messages
ParentLayout	Overview: Schedule · Homework Support: Credits · Live Classes · Messages
LearnerLayout	Learning: Course Status · Schedule* · Homework* · Exams* Achievements: Certificates · Tutoring* Communication: Messages (*school-only, gated by feature flags)

4. API / Data Layer, Env Vars & Socket.IO

Axios instance — `src/utils/api.ts`

```
const API_BASE_URL = import.meta.env.VITE_API_URL || 'https://quikskillsbackend.quikit.ai';
const api = axios.create({ baseURL: API_BASE_URL, headers: { 'Content-Type': 'application/json' } });

// Request: attach JWT; drop Content-Type for FormData (let browser set boundary)
api.interceptors.request.use(cfg => {
  const token = sessionStorage.getItem('access_token');
  if (token) cfg.headers.Authorization = `Bearer ${token}`;
  if (cfg.data instanceof FormData) delete cfg.headers['Content-Type'];
  return cfg;
});
// Response: on 401 clear session and redirect to /login?error=...
api.interceptors.response.use(r => r, err => {
  if (err.response?.status === 401) { sessionStorage.clear(); window.location.href = '/login?error=...'; }
  return Promise.reject(err);
});
```

Socket.IO — `src/utils/socket.ts`

Connects to `${VITE_API_URL}/messages` with `auth:{token}`, transports websocket+polling, infinite reconnection. `disconnectMessagesSocket()` is called on logout from every layout. The exam timer uses the `/exams` namespace.

Frontend Environment Variables

Variable	Purpose	Next.js equivalent
<code>VITE_API_URL</code>	REST + WebSocket base URL (dev <code>http://localhost:3020</code> , prod <code>https://quikskillsbackend.quikit.ai</code>)	<code>NEXT_PUBLIC_API_URL</code>

Migration note: `import.meta.env.VITE_*` → `process.env.NEXT_PUBLIC_*`. Browser `sessionStorage` / `localStorage` auth should move to **HTTP-only cookies** so Next.js middleware/server components can read auth.

5. State Management, Contexts & i18n

State is entirely **React Context** — no Redux/Zustand. Three providers wrap the app (`BrandingProvider` → `FeatureProvider` → app):

Context	Provides	Source
BrandingContext	primaryColor, secondaryColor, logo, setBranding(), refreshBranding(); applies CSS vars <code>--brand-primary/secondary</code>	<code>GET /tenants/branding/public</code> (pre-login) → <code>/tenants/current</code>
FeatureContext	tenantType (corporate/school), features{...16 flags...}, availableRoles, roleLabels, config, refresh()	<code>GET /tenants/current/features</code> ; re-fetches on login (storage event)
ThemeContext	activeTheme, draftTheme, isDarkMode, saveTheme(); light/dark/system; persisted in <code>localStorage.quickskill-theme</code>	OS <code>prefers-color-scheme</code> for system mode

Feature flags (FeatureContext): showCourses, showScormContent, showCompliance, showManagerDashboard, showBatches, showSchedule, showAttendance, showHomework, showCredits, showPayouts, showVideoClasses, showParentPortal, showMessaging, showCertificates, showAnalytics, showNotifications, showMultiLanguage. `FeatureRoute` component gates routes/menu items by these flags.

i18n (`src/i18n/config.ts`): i18next + react-i18next + language detector. Languages: English (en), Hindi (hi), Tamil (ta), Telugu (te), Kannada (kn), Marathi (mr), Bengali (bn). Detection: `localStorage.i18nextLng` → browser; fallback en. JSON dictionaries in `src/i18n/locales/` . `LanguageSwitcher` component toggles language.

6. Complete Pages Catalog (by Role)

Every page component under `src/pages/`, with purpose and the primary backend endpoints it calls. ~89 pages across 8 folders.

6.1 Authentication & Standalone (top-level)

Page	Purpose	Key APIs / Features
LoginPage	Email/password login + SSO entry	POST /auth/login; Google/Microsoft OAuth
AuthCallbackPage	OAuth SSO callback handler — stores token, routes to dashboard	OAuth provider redirect
SetupPasswordPage	First-time password setup from invite link	POST /auth/validate-setup-token, /auth/setup-password
ResetPasswordPage	Change password (in-app)	POST /auth/change-password
ProfileSettingsPage	Edit profile, photo, password, preferences	GET/PATCH /auth/profile, /auth/profile/upload-photo, /auth/change-password
VerifyCertificatePage	Public certificate verification by ID	GET /certificates/verify/:certificateId (public)
SuperAdminOnboarding	Initial super-admin onboarding wizard	tenant/feature bootstrap
ExamTakingPage	Fullscreen proctored exam interface (any role)	GET /exams/:id, exam-sessions/*, proctoring events, /exams WS timer
Dashboard / CoursesPage / CourseBuilderPage / UserManagementPage	Legacy/super-admin course & user management screens	/courses, /users, /auth/register
NotFoundPage	404 catch-all	—

6.2 SUPER_ADMIN (`pages/super-admin/`)

Page	Purpose	Key APIs
OrganizationsPage	Manage all tenant organizations	GET/POST/PATCH /tenants, /tenants/onboard
MasterLibraryPage	Browse global master course library	GET /courses/master/all
MasterLibraryBuilderPage	Author/edit master courses (drag-drop modules/lessons)	POST/PUT /courses, /modules, /lessons
CertificatesPage	Manage global certificate templates	GET/POST/DEL /certificates
ApprovalDashboardPage	Approve tenant-submitted courses & certificate templates	approval-workflow endpoints
SystemHealthPage	Platform health & usage overview	GET /tenants, /users, /audit/storage/global
AuditDashboardPage	Global activity logs & storage breakdown	GET /audit/activity-logs, /storage/global, /storage/tenants

6.3 TENANT_ADMIN & SUB_ADMIN (`pages/tenant-admin/` + `school-admin`)

Page	Purpose	Key APIs
TenantDashboardPage	Tenant home — KPIs, course progress	GET /analytics/corporate or school overview
LearnersPage	Corporate learner CRUD & assignment	GET /users, POST /auth/register, /admin-reset-password
CourseAssignmentPage	Assign courses to learners/groups	GET /course-assignments/courses, POST /assign
GroupsPage	Create/manage learner groups	GET/POST /groups, /groups/:id/members
CompliancePage	Compliance completion tracking & nudges	GET /compliance/analytics, /nudge-users, POST /nudge-all
CorporateAnalyticsPage	Corporate-wide analytics dashboard	GET /analytics/corporate
SchoolAnalyticsPage	School-wide analytics	GET /analytics/school
CourseAnalyticsPage	Per-course completion analytics	GET /analytics/corporate/course/:courseId

Page	Purpose	Key APIs
SubAdminsPage	Manage sub-administrators	/users (promote/revoke sub-admin)
SubAdminSubmissionsPage / TenantMySubmissionsPage	Approval queue & own submissions	approval-workflow endpoints
TenantCourseCreatorPage	Create courses (approval workflow)	POST /courses (submit for approval)
TenantCertificateTemplatesPage	Design tenant certificate templates	POST/PUT /certificates (jsPDF designer)
BrandingPage	Logo, colors, theme config	PATCH /tenants/:id/branding
StoragePage	Storage usage & quota	GET /tenants/usage, /audit/storage
TenantAuditPage	Tenant audit trail (+PDF export)	GET /tenant-audit/logs, /export-pdf
QuizProctoringReviewPage	Review proctored quiz incidents (corporate)	GET /quiz-proctoring/incidents/all, PATCH /:sid/incident
School-admin pages (pages/school-admin/)		
SchoolAdminDashboardPage	School operations home	school analytics, schedule
BatchesPage	Batch CRUD, teacher/student assignment	GET/POST /batches, /users?role=TEACHER, /academic-config/*
SchoolCourseAssignmentPage	Assign courses to batches	/course-assignments
TeachersPage / StudentsPage / ParentsPage	Manage teachers / students / parents	GET /users?role=..., bulk-upload, CRUD
PayoutsPage	Generate/approve teacher payouts	GET/POST/PATCH /payouts/*
CreditsConfigPage	Credit packages & allocation	GET/POST /credits/packages, /allocate
VideoConfigPage	Video provider configuration	PATCH /tenants/:id/video-config
SessionTimestampsPage / RecordingsPage	Session join logs & class recordings	GET /scheduling/admin/session-join-timestamps, /meetings/recordings
EscalationLogPage	Teacher-absence call escalations	GET /escalations/admin
NonTeachingTasksPage	Assign & approve non-teaching tasks	GET/POST/PATCH /non-teaching-work/*
TeacherLevelsPage	Teacher performance levels config	GET/POST /teacher-levels/*
TeacherAvailabilityPage	View/edit teacher availability	GET/PUT /teacher-availability/*
DemoAnalyticsPage	Demo/trial conversion analytics	GET /demo-analytics, /teacher-performance
QuestionBankPage	Exam question repository	GET/POST/PUT/DEL /question-bank
ExamListPage / ExamCreationPage	List & create exams	GET/POST /exams, /question-bank
ExamEvaluationPage	Grade exam submissions	GET /exam-sessions/exam/:id/submissions, PATCH /evaluate
IncidentReviewPage	Review exam proctoring incidents	GET /proctoring/exam/:id/incidents, PATCH /:sid/incident
ExamAnalyticsPage	Exam performance analytics	GET /exam-sessions/exam/:id/analytics
StudentExamDashboard	Admin view of student exam results	GET /exam-sessions/student/:id/results

6.4 MANAGER ([pages/manager/](#))

Page	Purpose	Key APIs
ManagerDashboardPage	Team overview & metrics	GET /manager/team-stats, /learner-courses
MyCoursesPage	Assigned courses & team progress	GET /course-assignments/my-assignments, /progress/my
MyTeamPage	Team roster & per-member status	GET /manager/team-list, /team/:id
CertificatesPage	Team member certificates	GET /certificates (team-filtered)

6.5 TEACHER ([pages/teacher/](#))

Page	Purpose	Key APIs
TeacherDashboardPage	Today's classes, tasks, earnings	GET /scheduling/teacher/classes, /analytics/teacher-dashboard

Page	Purpose	Key APIs
MyClassesPage	Batches, schedule, start video class	GET /scheduling/teacher/classes, /meetings, POST /meetings
AttendancePage	Mark class attendance	GET classes, POST /users/by-ids, /attendance/mark
HomeworkPage	Create & grade homework	GET/POST /homework, /upload/*, PATCH /submissions/:id/grade
PayoutSummaryPage	Earnings by month/year	GET /payouts/teacher?year=...
TeacherNonTeachingPage	Non-teaching task board	GET /non-teaching-work/teacher, PATCH /:id/complete
TeacherLevelPage	Personal performance level	GET /teacher-levels/me
TeacherAvailabilityPage	Set availability slots	GET/PUT /teacher-availability/me
TutoringRequestsPage	Accept/reject tutoring requests	GET /tutoring-requests/teacher, PATCH /accept, /reject, /complete

6.6 PARENT ([pages/parent/](#))

Page	Purpose	Key APIs
ParentDashboardPage	Child progress summary	GET /credits/my-balance, /scheduling/student/classes, /homework/student/submissions
ChildSchedulePage	Child's class schedule	GET /scheduling/student/classes
CreditsPage	View credits & transaction history	GET /credits/my-balance, /packages, /my-transactions
HomeworkStatusPage	Track child's homework	GET /homework/student/submissions
LiveClassStatusPage	Upcoming/live classes	GET /meetings/live-status/:studentId

6.7 LEARNER ([pages/learner/](#))

Page	Purpose	Key APIs
LearnerDashboardPage	Assigned courses, certificates, exams	GET /course-assignments/my-assignments, /certificates/my-certificates, /exams/student
CourseStatusTrackingPage	Per-course progress tracking	GET /course-assignments/my-assignments, /progress
CourseViewerPage	Fullscreen course player	POST /progress, /player/sync, quizzes
CompleteLessonPage	Lesson completion flow	POST /progress
StudentSchedulePage	Class schedule (school)	GET /scheduling/student/classes
LearnerHomeworkPage	View & submit homework (school)	GET /homework/student/submissions, POST /homework/:id/submit, /upload/*
TutoringRequestPage	Request tutoring sessions (school)	GET /tutoring-requests/available-teachers, POST /tutoring-requests
CertificatesPage	View & download certificates	GET /certificates/my-certificates, POST track-download

6.8 Shared ([pages/shared/](#))

Page	Purpose	Key APIs
MessagesPage	Chat hub (direct + group) for all roles	GET /messages/contacts, /conversations; Socket.IO /messages
VideoClassPage	Embedded video meeting (teacher/parent/learner)	GET /meetings/:meetingId; Jitsi/Zoom embed; join/leave tracking

7. Key Feature Components

Component	Role	What it does
CourseBuilder / CourseCreator / MasterCourseStudio	Admin	Drag-drop module/lesson authoring (@dnd-kit); SCORM/video/PDF lessons
QuizBuilderAdvanced / QuizCreator / PasteQuestionsModal	Admin/Teacher	Build quizzes (MCQ, T/F, fill-blank, match); bulk paste-import questions
CertificateDesigner	Admin	Visual certificate template editor; field placement; jsPDF render
UniversalLMSPlayer (learner/)	Learner	Main course player — SCORM/PPT (slides), PDF (scroll), Video/Audio (time); progress + offline
LockedCoursePlayer (learner/)	Learner	Legacy locked-course fallback player
CompliancePlayer	Learner	Mandatory compliance course playback & completion
VideoPlayer	Learner	react-player wrapper; position tracking; quiz overlays
ResourceUploader / SubModuleResourceEngine	Admin/Teacher	tus resumable multi-file upload; dynamic resource rendering
OnboardingWizard / TenantOnboardingWizard	Admin	Multi-step user & tenant setup (branding, features)
RoleSwitcher	Multi-role	Switch active role (updates <code>localStorage.activeRole</code>)
LanguageSwitcher	All	i18n language toggle
BulkUploadModal	Admin	CSV import of teachers/students/parents
AssignCourseModal / OnboardingWizard / NudgeBulkModal	Admin/Manager	Bulk assign, onboard, nudge
StorageHealthWidget	Admin	Storage quota indicator
UpgradeEmailTemplateEditor / WelcomeKitEditor	Super Admin	HTML email template editors (react-quill)
ErrorBoundary	All	Catches render errors, fallback UI + logs to <code>/logs/client-error</code>
FeatureRoute	All	Gates routes/menu by FeatureContext flags

Custom Hooks (`src/hooks/`)

Hook	Purpose
useFaceProctoring	MediaPipe face detection — no-face, multiple faces, looking away/down, eyes closed, too far; grace + debounce
useProctoringEngine	Exam behaviour monitor — tab switch, fullscreen exit, copy/paste, print, shortcuts → POST <code>/proctoring/sid/event</code>
useQuizProctoringEngine	Same for master-course quizzes → POST <code>/quiz-proctoring/sid/event</code>
useCertificateNotification	Toast/celebration when a certificate is issued
useOfflineDetection	Online/offline status → <code>OfflineAlert</code> ; local progress buffering
useAutoSave	Debounced autosave for builders/forms
useTenantOnboarding	Drives the tenant onboarding wizard state

8. Real-time & Media Features

- **Messaging (Socket.IO `/messages`)**: realtime send/receive, typing indicators, presence, reactions, group chat; auto-reconnect; disconnect on logout.
- **Exam timer (Socket.IO `/exams`)**: server-authoritative countdown — `timerSync` every 30s, warnings at 5min/1min, `forceSubmit` at timeout.
- **Face proctoring (MediaPipe)**: in-browser webcam analysis for quizzes/exams; violations logged server-side.
- **Behaviour proctoring**: fullscreen enforcement, tab/visibility, copy/paste/print/right-click/shortcut detection.
- **Video**: react-player (YouTube/Vimeo/HLS/MP4) for lessons; Jitsi/Zoom/Meet embeds for live classes (VideoClassPage); join/leave attendance tracking.
- **SCORM playback**: UniversalMSPlayer tracks `lesson_location` (slide index / scroll px / playback seconds), score, completion → `/player/sync` .
- **Resumable uploads**: tus-js-client for large media (pause/resume) → S3 via backend TUS endpoint.
- **PDF / canvas**: jsPDF + html2canvas for certificates & report export; canvas-confetti for completion celebrations; react-pdf for PDF lessons.
- **Offline**: useOfflineDetection buffers progress locally and syncs on reconnect.

9. Vite → Next.js Migration Guidance

Vite / React	Next.js (App Router)
<code>src/App.tsx</code> route table	File-based routes under <code>app/</code> ; route groups per role: <code>app/(super-admin)/</code> , <code>app/(tenant-admin)/</code> , <code>app/(teacher)/</code> , etc.
<code>components/layouts/*</code>	Per-group <code>layout.tsx</code> (sidebar + Outlet → children)
<code>PrivateRoute</code> / <code>RoleProtectedRoute</code>	<code>middleware.ts</code> (cookie-based auth + role check) + server-side redirects
<code>sessionStorage</code> / <code>localStorage</code> token	HTTP-only cookie for the JWT (readable in middleware/server components); keep theme/lang in <code>localStorage</code>
<code>BrandingProvider</code> / <code>FeatureProvider</code> / <code>ThemeProvider</code>	Client providers wrapping <code>app/layout.tsx</code> ; feature/branding can be fetched server-side per request for SSR theming
<code>src/utils/api.ts</code> (axios)	<code>lib/api.ts</code> client util, or Next.js Server Actions / Route Handlers for SSR fetches
<code>src/utils/socket.ts</code>	<code>lib/socket.ts</code> — client component only (<code>'use client'</code>)
Media/proctoring/quill/charts	Mark <code>'use client'</code> & <code>dynamic(() => ..., { ssr:false })</code> — they need <code>window</code> / <code>navigator</code>
<code>import.meta.env.VITE_API_URL</code>	<code>process.env.NEXT_PUBLIC_API_URL</code>
i18next browser detector	<code>next-intl</code> / <code>next-i18next</code> with locale routing or cookie

Key insight: The REST contract is unchanged by the backend's Mongo→Postgres switch, so the frontend migration is purely a **Vite→Next.js framework port**: convert the route table to the App Router, move auth from browser storage to cookies, and wrap browser-only components (`'use client'` / `ssr:false`). Business logic, API calls, contexts and Tailwind styling carry over directly.